

Drone Mapper — Assignment 3 High-Level Design

Authors: Sagi Eisenberg (207190406), Yoav Naaman (209543255)

TAU Advanced Topics in Programming (2026B). Assignment 3 splits the ex2 monolithic simulator into three separately built projects: a `simulator_207190406_209543255` executable that `dlopen`s `Algorithm_207190406_209543255.so` and `MissionControl_207190406_209543255.so`, then runs many missions in parallel under `-comparative` or `-competition` mode.

Artifact	Name
Executable	<code>simulator_207190406_209543255</code>
Algorithm plugin	<code>Algorithm_207190406_209543255.so</code>
MissionControl plugin	<code>MissionControl_207190406_209543255.so</code>

Plugin / UserCommon **namespaces** (code): `algorithm_207190406_209543255`, `mission_control_207190406_209543255`, `user_common_207190406_209543255`.

Folder responsibilities

Folder	Role	Build artifact
<code>common/</code>	Course-published interfaces used by more than one project. Read-only — we do not modify it.	none (INTERFACE lib <code>common::common</code>)
<code>Simulator/common_simulator/include/Simulator/</code>	Course-published interfaces used only by the Simulator	<code>Simulator/</code>
<code>Simulator/{include,src}/Simulator</code>	Course-published implementation	<code>simulator_207190406_209543255</code> executable
<code>MissionControl/common_mission_control/include/MissionControl/</code>	Course-published <code>IDroneControl</code>	<code>MissionControl/</code>

Folder	Role	Build artifact
MissionControl/{include,src}/	Our mission control implementation	MissionControl_207190406_209543255.so
Algorithm/{include,src}/	Our mapping algorithm	Algorithm_207190406_209543255.so
UserCommon/	Our code needed by more than one project. No build file — sources compile into each consumer.	none

The course staff placement rule: mocks and world simulation live in `Simulator/` because they would be replaced by real hardware APIs in production; mission control and the mapping algorithm live in their respective plugin projects. Shared cross-boundary interfaces stay in `common/`; simulator-only interfaces stay in `Simulator/common_simulator/`.

Main components

- **main (Simulator/src/main.cpp)** — Parses CLI via `parseSimulationCliArgs (SimulationCliArgs)`, creates the timestamped output directory with `createOutputDir`, loads the composition YAML, loads plugins through `PluginLoader`, builds one `SimulationRunFactoryImpl` per plugin binding, invokes `runPluginMatrix(bindings, composition, output_root, num_threads)` (`expandRunMatrix` runs inside that call), writes reports via `writeModeReport` → `writeComparativeReport` / `writeCompetitiveReport` plus per-plugin `writeSimulationOutputYaml`, then destroys plugin objects and calls `PluginLoader::unloadAll()` before return.
- **SimulationCli / SimulatorPaths (Simulator/io/SimulatorPaths.h)** — `parseSimulationCliArgs` returns `SimulationCliArgs`. Path helpers: `createOutputDir` (fresh `comparative_results_<UTC>` / `competition_<UTC>` folder) and `errorLogPathFromOutputMap` (derive `<plugin>_run_NNNN_error.log`).
- **YamlConfigParsers (Simulator/io/YamlConfigParsers.h)** — Five parsers: `parseCompositionFile`, `parseSimulationConfig`, `parseMissionConfig`, `parseDroneConfig`, `parseLidarConfig`. Parse failures are logged immediately through `RunErrorLog` / `IRunErrorLog`.

- **SimulatorReports** (`Simulator/io/SimulatorReports.h`) — `writeComparativeReport`, `writeCompetitiveReport`, `writeSimulationOutputYaml`. `writeModeReport` in `main.cpp` picks the mode writer.
- **PluginLoader** — `dlopen`s each `.so` once on the main thread (`loadAlgorithmSo`, `loadMissionControlsFromDirectory`, or the competitive-mode equivalents). Handles are `DlHandle (unique_ptr<void, DlCloser>)`. After each successful load it takes the factory registered by the plugin's static constructor into a `LoadedAlgorithmPlugin` / `LoadedMissionControlPlugin`. Access is `algorithmCount` / `algorithmAt` (and the mission-control equivalents) — no `algorithms()` vector getter. Never reloads a path already held open. `unloadAll()` drops factories then `dlclose`s every handle.
- **PluginRegistrar** — Simulator-owned singleton. `MappingAlgorithmRegistration` / `MissionControlRegistration` objects (defined in registration headers, `.cpp` in Simulator only) call `setPendingAlgorithmFactory` / `setPendingMissionControlFactory` during `dlopen`; the loader immediately `takePending*Factory()` so each registration is consumed exactly once. Factory aliases are `MappingAlgorithmFactory` / `MissionControlFactory`.
- **runPluginMatrix** — Free-function module (`RunMatrixOrchestrator.h`). Expands a composition into a flat `MatrixCell` list via `expandRunMatrix` (`simulation × mission × drone × lidar`) **inside** `runPluginMatrix(bindings, composition, output_root, num_threads)`, then runs every cell for every plugin binding. Pre-sizes the result table; each slot is written exactly once.
- **distributeWork** — Free-function scheduler. `num_threads` absent or 1 → main thread only. `N >= 2` → up to `N` worker threads (capped at cell count); main joins. Wraps per-cell work in `try/catch` so a plugin throw cannot terminate the process.
- **SimulationRunFactoryImpl** — Implements `simulator::ISimulationRunFactory`. Loads hidden and output `Map3DImpl` instances, constructs `MockGPS`, `MockLidar`, `MockMovement`, creates fresh plugin instances from the loaded factories, wires `MissionControlDependencies` / `MappingAlgorithmDependencies`, and returns a `SimulationRunImpl`.
- **SimulationRunImpl** — Implements `simulator::ISimulationRun`. Calls `missionControl->runMission()`, saves the output map, scores with `compareMaps(hidden, output, spawn)` against the hidden map, and returns `SimulationResult`. Per-run `RunErrorLog` path comes from `errorLogPathFromOutputMap`. Contains exceptions from `runMission()` so a single bad run scores -1 without aborting the matrix.
- **Mocks + maps + scoring** (`Simulator/src/`) — `Map3DImpl` (hidden + output), `MockGPS`, `MockLidar`, `MockMovement` (holds the hidden map;

throws on wall/boundary collision), `compareMaps`. `makeMap3D` is src-only (`Map3DNpy.h`).

- **MissionControlImpl_207190406_209543255** — Plugin entry point implementing `common::IMissionControl`. Creates `DroneControlImpl`, loops `step()` until the mission finishes or hits max steps, returns `MissionRunResult`.
- **DroneControlImpl** — Implements `mission_control::IDroneControl`. Each step queries GPS, asks the algorithm for a `MappingStepCommand`, executes movement or lidar scan, and writes scan results into the output map via `applyScanToMap`.
- **MappingAlgorithmImpl_207190406_209543255** — Plugin entry point implementing `common::IMappingAlgorithm`. Reads the world through `const common::IMap3D&` only. Uses `WavefrontPlanner` over a reachability substrate (`MappingAlgorithmFrontier`) to pick a cluster, then emits movement plus a score-aware scan toward that cluster. Scan templates are `ConeTemplateCache` / `VoxelStamp`.
- **SimulationCoordUtil** — Shared world/voxel helpers: `worldInitialDronePosition`, `forEachSphereSample`.
- **SimulationImpl** — Implements published `simulator::ISimulation`. Constructor takes one `ISimulationRunFactory&`, the plugin filename used in per-run map names, and `num_threads`. `run(composition, output_path)` delegates to `runPluginMatrix` for that single binding and returns `SimulationManagerReport`. Comparative/competitive CLI, plugin folders, and aggregate YAML still live in `main` — `ISimulation::run` cannot express mode or a plugin set.

Class diagram

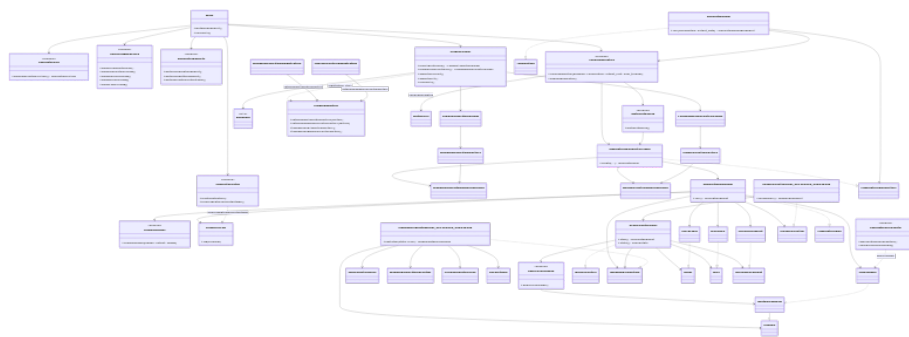


Figure 1: Class overview

```
classDiagram
    direction TB
```

```

class main {
    +writeModeReport()
    +unloadAll()
}

class SimulationCli {
    <<module>>
    +parseSimulationCliArgs() SimulationCliArgs
}

class SimulatorPaths {
    <<module>>
    +createOutputDir()
    +errorLogPathFromOutputMap()
}

class YamlConfigParsers {
    <<module>>
    +parseCompositionFile()
    +parseSimulationConfig()
    +parseMissionConfig()
    +parseDroneConfig()
    +parseLidarConfig()
}

class SimulatorReports {
    <<module>>
    +writeComparativeReport()
    +writeCompetitiveReport()
    +writeSimulationOutputYaml()
}

class runPluginMatrix {
    <<module>>
    +runPluginMatrix(bindings, composition, output_root, num_threads)
    +expandRunMatrix()
}

class distributeWork {
    <<module>>
    +distributeWork()
}

class compareMaps {
    <<module>>

```

```

    +compareMaps(hidden, output, spawn)
}

class applyScanToMap {
    <<module>>
    +applyScanToMap()
}

class PluginLoader {
    +loadAlgorithmSo() LoadedAlgorithmPlugin
    +loadMissionControlSo() LoadedMissionControlPlugin
    +algorithmCount()
    +algorithmAt()
    +unloadAll()
}

class DlHandle {
    <<RAII>>
}

class PluginRegistrar {
    +setPendingAlgorithmFactory(factory)
    +setPendingMissionControlFactory(factory)
    +takePendingAlgorithmFactory()
    +takePendingMissionControlFactory()
}

class MappingAlgorithmRegistration
class MissionControlRegistration
class MappingAlgorithmFactory
class MissionControlFactory
class MappingAlgorithmDependencies
class MissionControlDependencies
class MatrixCell
class LoadedAlgorithmPlugin
class LoadedMissionControlPlugin

class RunErrorLog {
    +log(ErrorRef)
}

class SimulationCoordUtil {
    <<module>>
    +worldInitialDronePosition()
    +forEachSphereSample()
}

```

```

class WavefrontPlanner
class MappingAlgorithmFrontier
class ConeTemplateCache
class VoxelStamp

class SimulationRunFactoryImpl {
    +create(...) ISimulationRun
}

class SimulationRunImpl {
    +run() SimulationResult
}

class SimulationImpl {
    +run(composition, output_path) SimulationManagerReport
}
class ISimulation

class Map3DImpl
class MockGPS
class MockLidar
class MockMovement

class MissionControlImpl_207190406_209543255 {
    +runMission() MissionRunResult
}

class DroneControlImpl {
    +step() DroneStepResult
    +state() DroneState
}

class MappingAlgorithmImpl_207190406_209543255 {
    +nextStep(state, scan) MappingStepCommand
}

class IMissionControl
class IDroneControl
class IMappingAlgorithm
class IMap3D
class IMutableMap3D
class ILidar
class IGPS
class IDroneMovement
class ISimulationRun

```

```

class ISimulationRunFactory

main --> SimulationCli
main --> SimulatorPaths
main --> YamlConfigParsers
main --> PluginLoader
main --> runPluginMatrix
main --> SimulatorReports
runPluginMatrix --> distributeWork
runPluginMatrix --> SimulationRunFactoryImpl
runPluginMatrix --> MatrixCell : expandRunMatrix
distributeWork --> SimulationRunFactoryImpl
SimulationRunFactoryImpl ..|> ISimulationRunFactory
SimulationRunFactoryImpl --> SimulationRunImpl
SimulationRunFactoryImpl --> MappingAlgorithmDependencies
SimulationRunFactoryImpl --> MissionControlDependencies
SimulationRunImpl ..|> ISimulationRun
SimulationRunImpl --> Map3DImpl
SimulationRunImpl --> MockGPS
SimulationRunImpl --> MockLidar
SimulationRunImpl --> MockMovement
SimulationRunImpl --> compareMaps
SimulationRunImpl --> RunErrorLog
SimulationRunImpl --> IMissionControl
SimulationRunImpl --> IMappingAlgorithm
SimulationImpl ..|> ISimulation
SimulationImpl --> runPluginMatrix
SimulationImpl --> ISimulationRunFactory
PluginLoader --> DlHandle
PluginLoader --> LoadedAlgorithmPlugin
PluginLoader --> LoadedMissionControlPlugin
PluginLoader ..> PluginRegistrar : registration ctors
MappingAlgorithmRegistration --> PluginRegistrar : setPendingAlgorithmFactory
MissionControlRegistration --> PluginRegistrar : setPendingMissionControlFactory
LoadedAlgorithmPlugin --> MappingAlgorithmFactory
LoadedMissionControlPlugin --> MissionControlFactory
MappingAlgorithmFactory --> MappingAlgorithmDependencies
MissionControlFactory --> MissionControlDependencies
MissionControlImpl_207190406_209543255 ..|> IMissionControl
MissionControlImpl_207190406_209543255 --> DroneControlImpl
DroneControlImpl ..|> IDroneControl
DroneControlImpl --> IMappingAlgorithm
DroneControlImpl --> ILidar
DroneControlImpl --> IGPS
DroneControlImpl --> IDroneMovement
DroneControlImpl --> applyScanToMap

```

```
applyScanToMap --> IMutableMap3D
MappingAlgorithmImpl_207190406_209543255 ..|> IMappingAlgorithm
MappingAlgorithmImpl_207190406_209543255 --> IMap3D
MappingAlgorithmImpl_207190406_209543255 --> WavefrontPlanner
MappingAlgorithmImpl_207190406_209543255 --> MappingAlgorithmFrontier
MappingAlgorithmImpl_207190406_209543255 --> ConeTemplateCache
MappingAlgorithmImpl_207190406_209543255 --> VoxelStamp
Map3DImpl ..|> IMutableMap3D
IMutableMap3D --|> IMap3D
MockLidar ..|> ILidar
MockGPS ..|> IGPS
MockMovement ..|> IDroneMovement
SimulationCoordUtil ..> Map3DImpl : world spawn
SimulatorPaths ..> RunErrorLog : errorLogPathFromOutputMap
```

Sequence: one comparative cell

Comparative mode fixes one algorithm `.so` and varies every `MissionControl` `.so` in a folder. `main` parses CLI, creates the output dir, parses the composition, loads plugins, then `runPluginMatrix` expands cells and `distributeWork` creates a fresh run per (plugin binding $\times$ matrix cell). After `runMission()` the output map is saved **before** `compareMaps`.

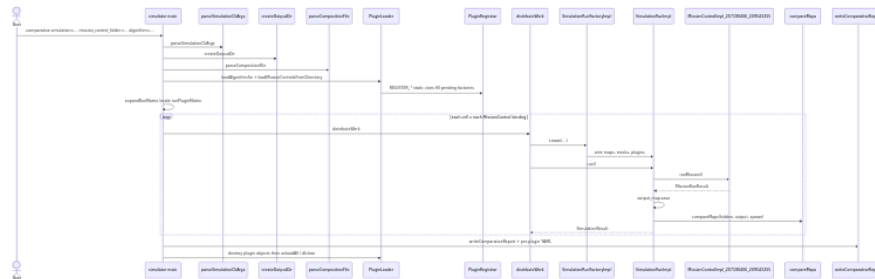


Figure 2: Comparative cell sequence

```
sequenceDiagram
    actor User
    participant Main as simulator main
    participant Cli as parseSimulationCliArgs
    participant Out as createOutputDir
    participant Comp as parseCompositionFile
    participant Loader as PluginLoader
    participant Reg as PluginRegistrar
    participant Dist as distributeWork
    participant Factory as SimulationRunFactoryImpl
    participant Run as SimulationRunImpl
```

```

participant MC as MissionControlImpl_207190406_209543255
participant Score as compareMaps
participant Rep as writeComparativeReport

User->>Main: -comparative simulation=... mission_control_folder=... algorithm=...
Main->>Cli: parseSimulationCliArgs
Main->>Out: createOutputDir
Main->>Comp: parseCompositionFile
Main->>Loader: loadAlgorithmSo + loadMissionControlsFromDirectory
Loader->>Reg: REGISTER_* static ctors fill pending factories
Main->>Main: expandRunMatrix inside runPluginMatrix
loop each cell x each MissionControl binding
    Main->>Dist: distributeWork
    Dist->>Factory: create(...)
    Factory->>Run: wire maps, mocks, plugins
    Dist->>Run: run()
    Run->>MC: runMission()
    MC-->>Run: MissionRunResult
    Run->>Run: output_map.save
    Run->>Score: compareMaps(hidden, output, spawn)
    Run-->>Dist: SimulationResult
end
Main->>Rep: writeComparativeReport + per-plugin YAML
Main->>Loader: destroy plugin objects then unloadAll / dlclose

```

Competition mode is the mirror image: one `MissionControl` .so is fixed and every algorithm in a folder is varied; the same `runPluginMatrix` / factory / run path applies (`writeCompetitiveReport` instead of `writeComparativeReport`).

Sequence: DroneControl step loop

Inside `MissionControlImpl_207190406_209543255::runMission()`, each iteration calls `DroneControlImpl::step()`. Each step invokes `nextStep` once and may execute an optional movement followed by at most one scan (then voxel fusion), matching the published movement → scan → fuse contract.

sequenceDiagram

```

participant MC as MissionControlImpl_207190406_209543255
participant DC as DroneControlImpl
participant Algo as MappingAlgorithmImpl_207190406_209543255
participant GPS as MockGPS
participant Move as MockMovement
participant Lidar as MockLidar
participant SR as applyScanToMap
participant Map as output Map3DImpl

```

```

loop until Finished / MaxSteps / Error

```

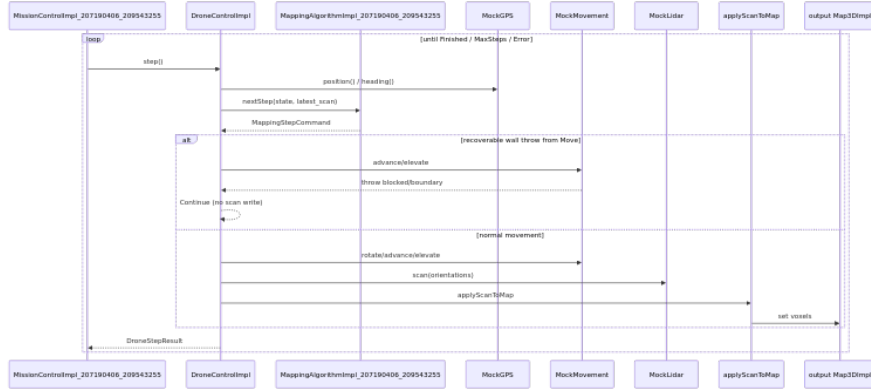


Figure 3: Drone step sequence

```

MC->>DC: step()
DC->>GPS: position() / heading()
DC->>Algo: nextStep(state, latest_scan)
Algo-->>DC: MappingStepCommand
alt recoverable wall throw from Move
  DC->>Move: advance/elevate
  Move-->>DC: throw blocked/boundary
  DC-->>DC: Continue (no scan write)
else normal movement
  DC->>Move: rotate/advance/elevate
  DC->>Lidar: scan(orientations)
  DC->>SR: applyScanToMap
  SR->>Map: set voxels
end
DC-->>MC: DroneStepResult
end

```

Recoverable collision handling: MockMovement detects wall/boundary collisions against the hidden map and throws (or returns a failure message containing `blocked` or `boundary`). DroneControlImpl catches these recoverable failures — both failed `MovementResult` and thrown `std::exception` — and returns `DroneStepStatus::Continue` without writing a scan for that step, allowing the algorithm to replan on the next iteration.

Backstop at the run boundary: Non-recoverable exceptions propagate out of `DroneControlImpl::step()` through `runMission()`. `SimulationRunImpl::run()` wraps the entire `runMission()` call in `try/catch`, logs the error, still saves the partial output map when possible, assigns score `-1`, and lets the run matrix continue.

Threading

CLI	Behavior
<code>num_threads</code> absent	Main thread runs all cells
<code>num_threads=1</code>	Main thread runs all cells
<code>num_threads=N</code> ($N \geq 2$)	Up to N worker threads plus the main thread (workers capped at cell count)

All `.so` files are loaded on the main thread before workers start. Each matrix cell creates fresh plugin instances via the stored factories — instances are never shared or cached between runs. The result table is pre-allocated; workers write disjoint indices with no mutex on the table itself. Aggregate YAML reports are written on the main thread after all workers join. `.so` handles are not `dlclose`d from worker threads.

Data and maps

Each run owns two `Map3DImpl` instances:

- **Hidden map** — Loaded from the simulation config `.npz`. Wired into `MockLidar` and `MockMovement` only. Never exposed to plugins.
- **Output map** — Created empty (or from mission resolution settings). Passed to `DroneControlImpl` as `IMutableMap3D`; mission control writes lidar fusion here.

The algorithm receives `const common::IMap3D&` through `MappingAlgorithmDependencies` and reads voxel occupancy for planning only — it never mutates the map. All scan writes go through `DroneControlImpl` → `applyScanToMap` → output `Map3DImpl`.

After the mission, `SimulationRunImpl` saves the output map then calls `compareMaps(hidden, output, spawn)` to score it. World <-> voxel conversion and axis offsets are centralized in `user_common_207190406_209543255::SimulationCoordUtil(worldInitialDronePosition, forEachSphereSample)`.